



KTH Engineering Sciences

Laboration 1

Ekvationslösning, linjär algebra och minstakvadratmetoden

I denna lab ska ni använda numeriska metoder för att lösa olinjära och linjära ekvationer samt minstakvadratproblem. Relaterat till detta ska ni studera konvergensfrågor, störningskänslighet och komplexitet. Laborationen består av fyra uppgifter. Ni ska implementera era lösningar i MATLAB.

Laborationen examineras med ett individuellt datorprov. Före provet ska ni skicka in era MATLAB-program i Canvas. Till uppgift 3 ska ni också skicka in en text-fil som innehåller den efterfrågade tabellen med exekveringstider. På provet ska ni lösa uppgifter som är snarlika uppgifterna nedan. Ni kommer då ha tillgång till de filer ni skickat in. Tänk därför på att skriva så tydlig och generell kod som möjligt, så att koden enkelt kan anpassas till andra problem utan att större delar behöver skrivas om.

Frågepunkterna (•) i uppgifterna är instuderingsfrågor som kopplar de empiriska resultaten till teorin i kursen. Genom att fundera på frågorna när ni håller på med laborationen blir det ofta lättare att förstå teorin. Frågorna behöver dock inte besvaras skriftligt eller lämnas in.

Se Canvas för mer instruktioner, tips och rekommendationer.

Det är inte tillåtet att använda AI-hjälpmedel i laborationen.
All kod måste vara helt människoproducerad.

1. Plankton - olinjär skalär ekvation

Följande ekvation¹ modellerar tillväxttakten (dP/dt) av plankton som funktion av totala populationen av plankton, P , som mäts i $\mu\text{g N/l}$.

$$\frac{dP}{dt} = rP \left(1 - \frac{P}{K}\right) - R_m Z \frac{P^2}{\alpha^2 + P^2} = f(P). \quad (1)$$

Uppgiften går ut på att bestämma samtliga nollställen till funktionen $f(P) = 0$ för värden på $P > 0$. (Bortse från den triviala roten $P = 0$.) Dessa motsvarar möjliga stationära tillstånd ($dP/dt = 0$) för planktonpopulationen.

Använd följande parametervärden:

$r = 0.3 \text{ [dag}^{-1}\text{]}$, $K = 108 \text{ [}\mu\text{g N/l]}$, $R_m = 0.7 \text{ [dag}^{-1}\text{]}$, $\alpha = 5.7 \text{ [}\mu\text{g N/l]}$ och $Z = 5 \text{ [}\mu\text{g N/l]}$.

Börja med att plotta $f(P)$ på ett lämpligt intervall för att få en uppfattning om hur många nollställen funktionen har då $P > 0$. Se till att du får med alla. Notera ungefärliga värden på nollställena och använd dem som startgissningar i metoderna nedan.

FIXPUNKTSMETODEN

Skriv ett MATLAB-program som beräknar nollställen för (1) med fixpunktsiterationen

$$P_{n+1} = P_n + 2rP_n \left(1 - \frac{P_n}{K}\right) - 2R_m Z \frac{P_n^2}{\alpha^2 + P_n^2}. \quad (2)$$

Programmet ska returnera ett svar med ett fel som är mindre än en given tolerans $\tau = 10^{-10}$. Använd lämpligt avbrottsvillkor för att säkerställa detta. Programmet ska också skriva ut P_n och skillnaden $|P_{n+1} - P_n|$ med minst 10 decimaler efter varje iteration. Notera: Programmet ska hitta så många nollställen som är möjligt med metoden. Det går dock inte att hitta samtliga. Programmet ska kunna hantera detta.

- Varför är (2) en fixpunktsiteration för $f(P) = 0$?
- Hur ska skillnaderna $|P_{n+1} - P_n|$ avta enligt teorin? Stämmer det i praktiken?
- Använd teorin för konvergens av fixpunktsmetoden för att visa vilka nollställen fixpunktsiterationen kan bestämma. Stämmer det med dina resultat?

NEWTONS METOD

Utöka programmet ovan så att det beräknar samtliga positiva nollställen till (1) med hjälp av Newtons metod. Programmet ska skriva ut samma saker som för fixpunktsmetoden och använda samma tolerans.

- Stämmer tumregeln för Newtons metod att antalet korrekta siffror dubblas i varje iteration?
- Jämför konvergens för fixpunktsiteration och Newtons metod. Vilken metod konvergerar snabbast? Hur hänger det ihop med metodernas konvergensordningar?

¹Ni kan läsa mer om modellen på sidan 985 i artikeln: J.E. Truscott and J. Brindley. Ocean plankton populations as excitable media. *Bulletin of Mathematical Biology*, 56:5:981-998, (1994).

Uppgift 1: På datorprovet förväntas du kunna använda dina MATLAB-program för att:

- Hitta nollställena till liknande ekvationer med liknande fixpunktsiterationer, med olika startgissningar och toleranser.
- Hitta nollställena till liknande ekvationer med Newtons metod, med olika startgissningar och toleranser.
- Empiriskt avgöra vilka nollställena fixpunktsiterationer kan konvergera till och hur snabb konvergensen då är.

2. Investeringsbeslut

Vi vill investera i ett projekt, vars årliga output har marknadspriset $p(t)$ MSEK vid tiden t . Priset varierar slumpmässigt utan trend med volatiliteten σ . Produktionskostnaden är w MSEK/år. För att investera i projektet behöver vi göra en grundinvestering på k MSEK. Att avveckla projektet medför en fast kostnad på ℓ MSEK. Vid vilket pris p_H bör vi då investera, och vid vilket pris p_L bör vi lägga ner projektet?

Om priset inte varierade, dvs om volatiliteten $\sigma = 0$, hade standardteorin givit svaret $p_H = w_H := w + rk$ och $p_L = w_L := w - r\ell$, där r är den riskfria räntan². På grund av volatiliteten kommer dock p_H ligga högre än så, och p_L lägre. För att bestämma p_H och p_L när $\sigma > 0$ ställer Dixit³ upp en modell som bygger på analogin med optionsprissättning och Black–Scholes ekvation. I modellen ges projektets förväntade nuvärde V vid ett givet pris p på formen

$$V(p) = \begin{cases} V_0(p) = ap^\beta, & \text{före investering,} \\ V_1(p) = bp^\alpha + \frac{p-w}{r}, & \text{efter investering.} \end{cases}$$

I uttrycken ovan är a, b parametrar som ska bestämmas (se nedan) och $\alpha < \beta$ är rötterna till andraderadekvationen

$$\frac{1}{2}\sigma^2 x(x-1) - r = 0.$$

Modellen ger sedan följande villkor för p_L och p_H ,

$$\begin{aligned} V_0(p_L) &= V_1(p_L) + \ell, \\ V_0(p_H) &= V_1(p_H) - k, \\ \frac{\partial V_0(p_L)}{\partial p} &= \frac{\partial V_1(p_L)}{\partial p}, \\ \frac{\partial V_0(p_H)}{\partial p} &= \frac{\partial V_1(p_H)}{\partial p}. \end{aligned}$$

Från dessa fyra ekvationer kan de fyra obekanta variablerna a , b , p_L och p_H bestämmas.

²Dvs, investera när priset överstiger produktions- och kapitalkostnaderna för investeringen och avveckla när priset understiger produktionskostnaden minus kapitalkostnaden för avvecklingen.

³A. Dixit. Entry and Exit Decisions under Uncertainty. *Journal of Political Economy*, 97(3):620–638, 1989.

NEWTONS METOD FÖR SYSTEM

a) *Förberedande arbete (tas även upp på Övning 2, 16/9)*

Samla de obekanta i en vektor $\mathbf{x} = (x_1, x_2, x_3, x_4) = (a, b, p_L, p_H)$. Skriv om ekvationerna ovan som ett system på formen $\mathbf{F}(x_1, x_2, x_3, x_4) = 0$ där $\mathbf{F}$ är en vektorvärd funktion med fyra komponenter. Beräkna jakobi-matrisen $\mathbf{J}$ till $\mathbf{F}$ med papper och penna. Notera att både $\mathbf{F}$ och $\mathbf{J}$ även beror på parametrarna σ, r, w, k och ℓ .

b) Skriv ett MATLAB-program som löser ekvationssystemet med Newtons metod för system. Felet ska vara mindre än 10^{-10} i alla komponenter. Programmet ska skriva ut skillnaderna $\|\mathbf{x}_{n+1} - \mathbf{x}_n\|_\infty$ med minst 10 decimaler i varje iteration. Använd programmet för att bestämma p_L och p_H för fallet $\sigma = 0.05, r = 0.03, w = 1, k = 3, \ell = 0.75$. Testa att variera parametrarna och verifiera att lösningarna beter sig korrekt. När σ minskas ska p_L och p_H närma sig w_L och w_H .

Tips:

- Definiera grundparametrarna σ, r, w, k och ℓ överst i programmet.
 - Beräkna därefter α och β från σ och r .
 - Implementera Newtons metod för system. Här behöver ni skriva kod som evaluerar $\mathbf{F}$ och $\mathbf{J}$ för ett givet $\mathbf{x}$ och parametrar $\alpha, \beta, r, w, k, \ell$.
 - Låt startgissningarna för p_H och p_L vara ca 5% högre, respektive lägre, än den teoretiska lösningen när $\sigma = 0$, (dvs w_H respektive w_L). Startgissningen för a, b kan vara $a = b = 1$.
- c) Lös problemet för volatiliteten σ i intervallet $[0.01, 0.15]$. Låt övriga parametrar vara oförändrade. Plotta lösningarna p_L och p_H som funktion av σ .
- d) Gå tillbaka till fallet $\sigma = 0.05$ men antag nu att var och en av parametrarna σ, r, w, k, ℓ har en osäkerhet på 1%. Bestäm den resulterande osäkerheten i p_L och p_H . Använd experimentell störningsanalys^a. För vilka variabler bidrar osäkerheten i indata mest respektive minst till osäkerheten i p_H ?

Ledning: Skriv om programmet i deluppgift (b) till en funktion som tar σ, r, w, k, ℓ som argument och returnerar p_H och p_L . Anropa funktionen högst 6 gånger för att få fram osäkerheten.

^aSe anteckningar om Fel- och störningsanalys i Canvas.

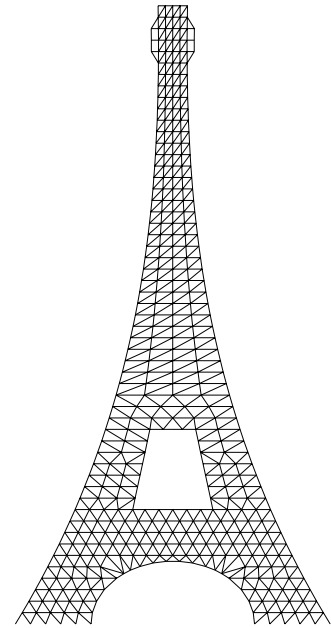
Uppgift 2: På datorprovet förväntas du kunna använda dina MATLAB-program för att:

- Lösa investeringsproblemet för andra parametervärden.
- Producera liknande plottar av lösningsvärden som funktion av en parameter.
- Genomföra experimentell störningsanalys med liknande antaganden om parametrar och osäkerheter i indata.

3. Stora matriser

I många tillämpningar måste man lösa *stora* linjära ekvationssystem, med miljontals obekanta. Det är i dessa fall som effektiva algoritmer blir viktiga att använda. Som exempel ska ni här räkna på ett komplicerat fackverk: en modell av Eiffeltornet. Ett fackverk består av stänger förenade genom leder i ett antal noder. Ni ska beräkna deformationen av fackverket när noderna belastas av yttre krafter. Ekvationerna för deformationen härleds i hållfasthetsläran, och baseras på att förskjutningarna i varje nod är små, och att Hookes lag gäller för förlängningen av varje stång. Ett dokument med härledningar finns i Canvas.

I slutändan får man ett linjärt ekvationssystem på formen $A\mathbf{x} = \mathbf{b}$. När antalet noder i fackverket är n kommer antalet obekanta vara $2n$ och $A \in \mathbb{R}^{2n \times 2n}$. Matrisen A brukar kallas *styvhetsmatrisen*.



Högerledet $\mathbf{b}$ innehåller de givna yttre krafterna som verkar på noderna,

$$\mathbf{b} = (F_1^x, F_1^y, F_2^x, F_2^y, \dots, F_n^x, F_n^y)^T, \quad \mathbf{b} \in \mathbb{R}^{2n},$$

där $\mathbf{F}_j = (F_j^x, F_j^y)^T$ är kraften i nod j . Lösningen $\mathbf{x}$ innehåller de resulterande (obekanta) förskjutningarna,

$$\mathbf{x} = (\Delta x_1, \Delta y_1, \Delta x_2, \Delta y_2, \dots, \Delta x_n, \Delta y_n)^T, \quad \mathbf{x} \in \mathbb{R}^{2n}.$$

Här är alltså $(\Delta x_j, \Delta y_j)^T$ förskjutningen av nod j när fackverket belastas med krafterna i $\mathbf{b}$.

I Canvas finns filerna `eiffel1.mat`, `eiffel2.mat`, ... `eiffel6.mat`. De innehåller sex olika modeller av Eiffeltornet med växande detaljrikedom ($n = 261, 399, 561, 1592, 2290, 4330$). Varje modell består av nodkoordinater i vektorerna `xnod`, `ynod`, stångindex i matrisen `bars` (används bara för plottningen) och styvhetsmatrisen `A`.

LINJÄRA EKVATIONSSYSTEM

a) Förberedande arbete

Ladda in den minsta modellen `eiffel1.mat` i MATLAB med kommandot `load`. Hämta funktionsfilen `trussplot.m` från Canvas och anropa den med `trussplot(xnod,ynod,bars)` för att plotta tornet. Välj en av noderna och belasta den med en kraft snett uppåt vänster med beloppet $\sqrt{2}$. (Sätt $F_j^x = -1$ och $F_j^y = 1$ för något j . Låt resten av elementen i $\mathbf{b}$ vara noll, dvs `b=zeros(2*n,1); b(j*2-1)=-1; b(j*2)=1;` i MATLAB.) Lös systemet $A\mathbf{x} = \mathbf{b}$ med backslash för att få fram förskjutningarna i alla noder. Beräkna de nya koordinaterna för det belastade tornet, $x_i^{\text{bel}} = x_i + \Delta x_i$, etc.:

```
xbel = xnod + x(1:2:end); ybel = ynod + x(2:2:end);
```

Plotta det belastade tornet. Använd `hold on` för att plotta de två tornen ovanpå varandra i samma figur. Markera vilken nod ni valt med en asterisk `*` i figuren.

TIDSKOMPLEXITET

- b) Backslash-kommandot i MATLAB använder normalt vanlig gausseliminering. Undersök hur tidsåtgången för gausseliminering beror på systemmatrisens storlek genom att lösa ekvationssystemet $A\mathbf{x} = \mathbf{b}$ med ett godtyckligt valt högerled $\mathbf{b}$ (tex med $\mathbf{b}=\text{randn}(N,1)$) för var och en av de sex modellerna. Använd MATLAB-kommandona `tic` och `toc` (`help tic` ger mer info).^a Plotta tidsåtgången mot antal obekanta $N = 2n$ i en `loglog`-plot; använd också `grid on`.
- Hur ska tidsåtgången bero på N enligt teorin? Stämmer det överens med din plot?

LU-FAKTORISERING OCH GLESA MATRISER

- c) Ni ska nu räkna ut vilken nod i fackverket som är mest känslig för belastning snett uppåt vänster. Tidsåtgången för ett par olika metoder ska jämföras. För att hitta den sökta noden måste man lösa ekvationssystemet en gång för varje nod. Mer precist behöver man bestämma $\mathbf{x}_j$ så att

$$A\mathbf{x}_j = \mathbf{b}_j, \quad j = 1, \dots, n,$$

där $\mathbf{b}_j$ betecknar högerledsvektorn som motsvarar belastning snett uppåt vänster, dvs som i deluppgift (a), $F_j^x = -1$, $F_j^y = 1$ och alla övriga element noll. Man definierar sedan den totala förskjutningen T_j som den euklidiska normen av $\mathbf{x}_j$,

$$T_j := \|\mathbf{x}_j\|, \quad \|\mathbf{x}\| = \left(\sum_{k=1}^n \Delta x_k^2 + \Delta y_k^2 \right)^{1/2}.$$

Den känsligaste noden är den som ger störst T_j .

- Skriv ett MATLAB-program som loopar igenom alla noderna i fackverket, beräknar $\mathbf{x}_j$ och T_j samt hittar den känsligaste noden. Använd backslash här för att lösa ekvationssystemen och kommandot `norm` för att beräkna T_j . Förvissa er om rimligheten i resultatet genom att tex plotta tornet och markera den känsligaste noden i plotten. (Testkör med minsta modellen `eiffel1.mat`.)
- Lägg till kod som mäter tiden för beräkningarna. Använd `tic` och `toc` som tidigare. Inkludera `inte` plottning och inladdning av filer i tidsmätningen.
- Den naiva metoden att lösa ekvationssystemen med backslash varje gång blir mycket tidskrävande för de större modellerna. Skriv istället kod som använder LU-faktorisering för att lösa samma linjära ekvationssystem för många högerled, och anpassa den till det här problemet. Använd MATLAB-kommandot `lu`. LU-faktoriseringen ska bara göras en gång, utanför loopen, men den ska inkluderas i tidsmätningen. Verifiera att metoden ger samma resultat som den naiva metoden och att den är snabbare.
- När en matris är gles kan man lösa ekvationssystemet med effektivare metoder än standard gausseliminering. (Använd kommandot `spy(A)` för att förvissa dig om att A i det här fallet faktiskt är gles (bandad).) Konvertera A till glest format med kommandot `A=sparse(A)`. (Inkluderas `inte` i tidsmätningen!) MATLAB använder då bättre metoder när backslash och `lu` anropas. Verifiera att beräkningarna går snabbare och att du fortfarande får samma resultat.

Gå igenom de sex modellerna `eiffel1.mat`, ..., `eiffel6.mat` och mät hur lång tid det tar att lösa problemet med de fyra olika metoderna som diskuterats ovan.^b Sammanfatta resultaten i en tabell enligt nedan. Tabellen ska skickas in.

	Naiv	LU	Gles (ej LU)	Gles+LU
<code>eiffel1.mat</code>				
<code>eiffel2.mat</code>				
$\vdots$				
<code>eiffel6.mat</code>				

- Varför går det snabbare att lösa problemet med LU-faktorisering?
- Vilken metod löser problemet snabbast? (Med/utan LU? Full/gles lösare?)
- För vilken modell blir tidsvinsten störst? Varför?

^aFör att få bra noggrannhet i tidsmätningen (speciellt om den är kort) kan man mäta totaltiden för flera upprepade beräkningar, och sedan dividera tiden med antalet upprepningar.

^bNi kan hoppa över de fall som tar mer än en minut.

Uppgift 3: På datorprovet förväntas du kunna använda dina MATLAB-program för att:

- ▶ Mäta och plotta tidsåtgången för andra följder av linjära ekvationssystem.
- ▶ Bestämna känsligaste noden för andra Eiffeltornmodeller (matriser) och andra applicerade krafter F_j^x, F_j^y .
- ▶ Använda LU-faktorisering med glesa matriser för att lösa andra problem där man har samma systemmatris men många olika högerled.
- ▶ Avgöra vilka metoder som är snabbare än andra baserat på värdena i tabellen och/eller nya körningar.

4. Sveriges befolkningsmängd

På kurshemsidan finns filen `swedpop.mat` som innehåller data om Sveriges befolkningsmängd från 1900 till 2024. Er uppgift är att anpassa ett par matematiska modeller till den givna datan med minstakvadratmetoden.

Läs in filen i MATLAB med kommandot `load`. Befolkningsmängderna och motsvarande tidpunkter (mätt i miljoner människor respektive år sedan 1900) finns i variablerna `Y` och `T`. Nedan används notationen y_i och t_i för elementen i vektorerna.

MINSTAKVADRATMETODEN

- a) Antag först att befolkningsmängden kan beskrivas med en kvadratisk funktion. Anpassa därför data till modellen

$$y(t) = c_0 + c_1 t + c_2 t^2,$$

genom att ställa upp och lösa lämpligt linjärt ekvationssystem för c_0 , c_1 och c_2 . Skriv ut värdena på koefficienterna c_0 , c_1 och c_2 på skärmen med 10 decimaler. Plotta data och den anpassade kvadratiske modellen i samma figur. Ser anpassningen bra ut?

Residualvektorn $\mathbf{R}$ har elementen $r_i = y_i - y(t_i)$. Beräkna dess norm $\|\mathbf{R}\|_2$ med `norm(R)` och skriv ut värdet på skärmen.

- b) Vi vill förbättra modellen. Plotta därför residualerna r_i för den kvadratiske modellen som funktion av tiden. Den verkar ha ett cykliskt beteende. Vi ansätter därför den förfinade modellen

$$y(t) = d_0 + d_1 t + d_2 t^2 + d_3 \sin(kt) + d_4 \cos(kt).$$

Uppskatta periodtiden L i residualplotten, beräkna motsvarande k och anpassa denna nya modell till datan. Plotta resultatet som ovan. Skriv ut värdena på koefficienterna och residualvektorns norm. Är anpassningen bättre?

- c) Låt nu även k vara en okänd parameter. Modellen blir då olinjär. Hitta hela parameteruppsättningen $d_0, d_1, d_2, d_3, d_4, k$ som ger bäst approximation av data i minstakvadratmening. Använd Gauss-Newtons metod med resultatet från deluppgift (b) som startgissning. Skriv ut värdena på parametrarna, inklusive k (och motsvarande L), och residualvektorns norm.

Plotta slutligen data och alla de tre modellernas anpassningar i samma figur. Jämför residualnormerna för modellerna. Vilken är störst/minst? Varför?

- d) Använd modellerna för att uppskatta hur stor befolkningsmängden kommer vara år 2050. Jämför med SCBs prognos på 11.29 miljoner.^a Skriv ut dina tre prognoser.

^a <https://www.scb.se/hitta-statistik/sverige-i-siffror/manniskorna-i-sverige/befolkningsprognos-for-sverige/>

På datorprovet förväntas du kunna använda dina MATLAB-program för att:

- Anpassa liknande linjära och olinjära modeller till liknande tidsserier med minstakvadratmetoden och Gauss-Newtons metod.
- Hitta startgissningar till Gauss-Newtons metod för liknande olinjära modeller.
- Göra liknande prognoser för värden utanför det givna tidsintervallet.